

bok25

基
礎
班

、**考研数据結構**



交换排序

冒泡排序



交换排序

冒泡排序

基本思想

从前往后依次比较相邻元素，如果逆序则交换，不断循环到有序

从后往前也行

1号元素与2号元素比较（交换），2号与3号.....n-1与n号元素比较（交换）称为第一趟冒泡排序

第一趟冒泡排序后，关键字最大的元素被安置在最后一个记录的位置上

再进行第二趟，第三趟.....直到整体有序



交换排序

冒泡排序

基本思想

从前往后依次比较相邻元素，如果逆序则交换，不断循环到有序

从后往前也行

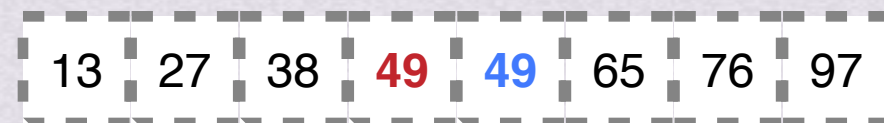
```
void BubbleSort(SqList &L)
{
    // 对顺序表 L 进行冒泡排序
    m=L.length-1;
    flag=1; // flag 置为 0，如果本趟排序没有发生交换，则不会执行下一趟排序
    while((m>0)&&(flag==1))
    {
        flag=0; // flag用来标记某一趟排序是否发生交换
        for(j=1; j<=m; j++)
        {
            if(L[j]>L[j+1])
            {
                flag=1; // flag 置为 1，表示本趟排序发生了交换
                t=L[j];
                L[j]=L[j+1];
                L[j+1]=t; // 交换前后两个记录
            } // if
        } // for
        --m;
    } // while
} // BubbleSort
```

平均时间复杂度 $O(n^2)$

空间复杂度 $O(1)$

稳定排序

也适用于链式存储



已有序元素



交换排序

冒泡排序

基本思想

从前往后依次比较相邻元素，如果逆序则交换，不断循环到有序

从后往前也行

```
void BubbleSort(SqList &L)
{
    // 对顺序表 L 进行冒泡排序
    m=L.length-1;
    flag=1; // flag 置为 0，如果本趟排序没有发生交换，则不会执行下一趟排序
    while((m>0)&&(flag==1))
    {
        flag=0; // flag用来标记某一趟排序是否发生交换
        for(j=1; j<=m; j++)
        {
            if(L[j]>L[j+1])
            {
                flag=1; // flag 置为 1，表示本趟排序发生了交换
                t=L[j];
                L[j]=L[j+1];
                L[j+1]=t; // 交换前后两个记录
            } // if
        } // for
        --m;
    } // while
} // BubbleSort
```

1. 设立一个flag记录本轮是否发生交换

m记录仍无序的元素个数



交换排序

冒泡排序

基本思想

从前往后依次比较相邻元素，如果逆序则交换，不断循环到有序

从后往前也行

```
void BubbleSort(SqList &L)
{
    // 对顺序表 L 进行冒泡排序
    m=L.length-1;
    flag=1; // flag 置为 0，如果本趟排序没有发生交换，则不会执行下一趟排序
    while((m>0)&&(flag==1))
    {
        flag=0; // flag用来标记某一趟排序是否发生交换
        for(j=1; j<=m; j++)
        {
            if(L[j]>L[j+1])
            {
                flag=1; // flag 置为 1，表示本趟排序发生了交换
                t=L[j];
                L[j]=L[j+1];
                L[j+1]=t; // 交换前后两个记录
            } // if
        } // for
        --m;
    } // while
} // BubbleSort
```

1. 设立一个flag记录本轮是否发生交换

m记录仍无序的元素个数

2. 在m仍大于0时，flag=1（本轮发生交换）时
不断进行以下操作：

2-1 将flag置0，开始新一轮冒泡

2-2 判断当前元素和下一个元素大小

如果逆序则交换，并且置flag=1



交换排序

冒泡排序

```
void BubbleSort(SqList &L)  传入顺序表
{
    // 对顺序表 L 进行冒泡排序
    m=L.length-1;
    flag=1; //flag 置为 0, 如果本趟排序没有发生交换, 则不会执行下一趟排序
    while((m>0)&&(flag==1))
    {
        flag=0; //flag用来标记某一趟排序是否发生交换
        for(j=1; j<=m; j++)
        {
            if(L[j]>L[j+1])
            {
                flag=1; //flag 置为 1, 表示本趟排序发生了交换
                t=L[j];
                L[j]=L[j+1];
                L[j+1]=t; //交换前后两个记录
            } //if
        } //for
        m--;
    } //while
} //BubbleSort
```

存储内容	-	15	27	29	38
数组下标	0	1	2	3	4



交换排序

冒泡排序

基本思想

从前往后依次比较相邻元素，如果逆序则交换，不断循环到有序

从后往前也行

```
void BubbleSort(SqList &L)
{
    // 对顺序表 L 进行冒泡排序
    m=L.length-1;
    flag=1; // flag 置为 0，如果本趟排序没有发生交换，则不会执行下一趟排序
    while((m>0)&&(flag==1))
    {
        flag=0; // flag用来标记某一趟排序是否发生交换
        for(j=1; j<=m; j++)
        {
            if(L[j]>L[j+1])
            {
                flag=1; // flag 置为 1，表示本趟排序发生了交换
                t=L[j];
                L[j]=L[j+1];
                L[j+1]=t; // 交换前后两个记录
            } // if
        } // for
        --m;
    } // while
} // BubbleSort
```

平均时间复杂度 $O(n^2)$

空间复杂度 $O(1)$

稳定排序

也适用于链式存储



交换排序

快速排序



交换排序

快速排序

针对冒泡排序的改进

基本思想

1. 选择一个元素作为枢纽（选择表中第一个元素）
2. 让枢纽左边存储小于枢纽的元素，右边存储大于枢纽的元素

具体做法：表左右侧设立low,high指针，初始low=1;high=L.length

2-1. 从表最右侧向左搜索，找到第一个小于枢纽关键字的记录，移到low处

2-2. 从表的最左侧位置，向右搜索到第一个关键字大于枢纽的记录和枢纽交换

重复2-1及2-2,直到low==high，此时low或high的位置即枢纽在此趟排序中的最终位置
原表被分为两个子表



交换排序

快速排序

针对冒泡排序的改进

基本思想

1. 选择一个元素作为枢纽（选择表中第一个元素）
2. 让枢纽左边存储小于枢纽的元素，右边存储大于枢纽的元素

具体做法：表左右侧设立low,high指针，初始low=1;high=L.length

high指针往左（小）走，碰到比枢纽元素小的就把元素放到low位置，然后操作low指针

low指针往右（大）走，碰到比枢纽元素大的就把元素放到high位置，然后操作high指针

交换排序快速排序

```
void QSort(SqList &L, int low, int high)
{
    //调用前置初值 : low=1; high=L.length;
    // 对顺序表 L 中的子表 L[low..high] 进行快速排序
    if(low < high)
    {
        //长度大于1
        pivotloc = Partition(L, low, high);
        //将L[low..high]一分为二, pivotloc是枢轴位置
        QSort(L, low, pivotloc-1); //对左子表递归排序
        QSort(L, pivotloc+1, high); //对右子表递归排序
    }
}
```

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
    L[0] = L[low]; //用子表的第一个记录作为枢轴记录
    pivotkey = L[low]; //枢轴记录关键字保存在pivotkey中
    while(low < high) //从表的两端交替地向中间查找
    {
        while(low < high && L[high] >= pivotkey) --high;
        L[low] = L[high]; // 将比枢轴记录小的记录移到低端
        while(low < high && L[low] <= pivotkey) ++low;
        L[high] = L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low] = L[0]; //枢轴记录到位
    return low; //返回枢轴位置
}
```

存储内容	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8

交换排序快速排序

```
void QSort(SqList &L,int low,int high)
{
    //调用前置初值 :low=1; high=L.length;
    // 对顺序表 L 中的子表 L[low..high] 进行快速排序
    if(low<high)
    {
        //长度大于1
        pivotloc=Partition(L, low, high);
        //将L[low..high]一分为二, pivotloc是枢轴位置
        QSort(L, low, pivotloc-1); //对左子表递归排序
        QSort(L, pivotloc+1, high); //对右子表递归排序
    }
}
```

- 1.low>=high即排序完成，在未排序完成前，进行以下操作：
 - 1-1对无序表进行一次快排，返回本次的枢纽下标pivotloc
 - 1-2 对无序表从最左到pivotloc-1递归的进行完整快速排序
 - 1-3 对无序表从pivotloc+1到最右边递归的进行完整快速排序

存储内容	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8

交换排序快速排序

```
void QSort(SqList &L,int low,int high)
{
    //调用前置初值 :low=1; high=L.length;
    // 对顺序表 L 中的子表 L[low..high] 进行快速排序
    if(low<high)
    {
        //长度大于1
        pivotloc=Partition(L, low, high);
        //将L[low..high]一分为二, pivotloc是枢轴位置
        QSort(L, low, pivotloc-1); //对左子表递归排序
        QSort(L, pivotloc+1, high); //对右子表递归排序
    }
}
```

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
    L[0]=L[low]; //用子表的第一个记录作为枢轴记录
    pivotkey=L[low]; //枢轴记录关键字保存在pivotkey中
    while(low<high) //从表的两端交替地向中间查找
    {
        while(low<high&&L[high]>=pivotkey) --high;
        L[low]=L[high]; // 将比枢轴记录小的记录移到低端
        while(low<high&&L[low]<=pivotkey) ++low;
        L[high]=L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low]=L[0]; //枢轴记录到位
    return low; //返回枢轴位置
}
```

存储内容	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8



交换排序快速排序

一趟快速排序的操作代码：

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序，返回枢轴位置
    L[0]=L[low]; // 用子表的第一个记录作为枢轴记录
    pivotkey=L[low]; // 枢轴记录关键字保存在pivotkey中
    while(low<high) // 从表的两端交替地向中间查找
    {
        while(low<high&&L[high]>=pivotkey) --high;
        L[low]=L[high]; // 将比枢轴记录小的记录移到低端
        while(low<high&&L[low]<=pivotkey) ++low;
        L[high]=L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low]=L[0]; // 枢轴记录到位
    return low; // 返回枢轴位置
}
```

存储内容	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8



交换排序快速排序

一趟快速排序的操作代码：

1.将第一个元素作为枢轴记录
并且保存在pivotkey中

```
int Partition(SqList &L, int low, int high)
{ // 对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
  L[0]=L[low]; // 用子表的第一个记录作为枢轴记录
  pivotkey=L[low]; // 枢轴记录关键字保存在pivotkey中
  while(low<high) // 从表的两端交替地向中间查找
  {
    while(low<high&&L[high]>=pivotkey) --high;
    L[low]=L[high]; // 将比枢轴记录小的记录移到低端
    while(low<high&&L[low]<=pivotkey) ++low;
    L[high]=L[low]; // 将比枢轴记录大的记录移到高端
  } //while
  L[low]=L[0]; // 枢轴记录到位
  return low; // 返回枢轴位置
}
```

存储内容	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8



交换排序快速排序

一趟快速排序的操作代码：

1.将第一个元素作为枢轴记录
并且保存在pivotkey中

只要low<high，就要进行快排操作

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序，返回枢轴位置
    L[0]=L[low]; // 用子表的第一个记录作为枢轴记录
    pivotkey=L[low]; // 枢轴记录关键字保存在pivotkey中
    while(low<high) // 从表的两端交替地向中间查找
    {
        while(low<high&&L[high]>=pivotkey) --high;
        L[low]=L[high]; // 将比枢轴记录小的记录移到低端
        while(low<high&&L[low]<=pivotkey) ++low;
        L[high]=L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low]=L[0]; // 枢轴记录到位
    return low; // 返回枢轴位置
}
```

存储内容

-	49	38	65	97	76	13	27	49
---	----	----	----	----	----	----	----	----

数组下标

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

交换排序快速排序

一趟快速排序的操作代码:

1.将第一个元素作为枢轴记录
并且保存在pivotkey中

只要low<high, 就要进行快排操作

2.视角从high指针开始:

如果high指向的元素值大于pivotkey, high指针就前移
直到发现high指向的元素值小于pivotkey, 将该元素
放在low指针指向的位置, 并切换到low指针视角

如果low指向的元素值小于pivotkey, low指针就后移
直到发现low指向的元素值大于pivotkey, 将该元素
放在high指针指向的位置, 并切换回high指针视角

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
    L[0]=L[low]; //用子表的第一个记录作为枢轴记录
    pivotkey=L[low]; //枢轴记录关键字保存在pivotkey中
    while(low<high) //从表的两端交替地向中间查找
    {
        while(low<high&&L[high]>=pivotkey) --high;
        L[low]=L[high]; // 将比枢轴记录小的记录移到低端
        while(low<high&&L[low]<=pivotkey) ++low;
        L[high]=L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low]=L[0]; //枢轴记录到位
    return low; //返回枢轴位置
}
```

存储内容	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8

交换排序快速排序

一趟快速排序的操作代码:

1.将第一个元素作为枢轴记录
并且保存在pivotkey中

在low<high时，一趟快速排序尚未完成

2.视角从high指针开始:
如果high指向的元素值大于pivotkey, high指针就前移
直到发现high指向的元素值小于pivotkey, 将该元素
放在low指针指向的位置, 并切换到low指针视角

如果low指向的元素值小于pivotkey, low指针就后移
直到发现low指向的元素值大于pivotkey, 将该元素
放在high指针指向的位置, 并切换回high指针视角

3.将枢轴元素放到low指针指向的位置, 并且返回枢轴的下标

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
    L[0]=L[low]; // 用子表的第一个记录作为枢轴记录
    pivotkey=L[low]; // 枢轴记录关键字保存在pivotkey中
    while(low<high) // 从表的两端交替地向中间查找
    {
        while(low<high&&L[high]>=pivotkey) --high;
        L[low]=L[high]; // 将比枢轴记录小的记录移到低端
        while(low<high&&L[low]<=pivotkey) ++low;
        L[high]=L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low]=L[0]; // 枢轴记录到位
    return low; // 返回枢轴位置
}
```

存储内容	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8

交换排序快速排序

```
void QSort(SqList &L,int low,int high)
{
    //调用前置初值 :low=1; high=L.length;
    // 对顺序表 L 中的子表 L[low..high] 进行快速排序
    if(low<high)
    {
        //长度大于1
        pivotloc=Partition(L, low, high);
        //将L[low..high]一分为二, pivotloc是枢轴位置
        QSort(L, low, pivotloc-1); //对左子表递归排序
        QSort(L, pivotloc+1, high); //对右子表递归排序
    }
}
```

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
    L[0]=L[low]; //用子表的第一个记录作为枢轴记录
    pivotkey=L[low]; //枢轴记录关键字保存在pivotkey中
    while(low<high) //从表的两端交替地向中间查找
    {
        while(low<high&&L[high]>=pivotkey) --high;
        L[low]=L[high]; // 将比枢轴记录小的记录移到低端
        while(low<high&&L[low]<=pivotkey) ++low;
        L[high]=L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low]=L[0]; //枢轴记录到位
    return low; //返回枢轴位置
}
```

存储内容	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8



交换排序快速排序

```
void QSort(SqList &L, int low, int high)
{
    //调用前置初值 : low=1; high=L.length;
    // 对顺序表 L 中的子表 L[low..high] 进行快速排序
    if (low < high)
    {
        //长度大于1
        pivotloc = Partition(L, low, high);
        //将L[low..high]一分为二, pivotloc是枢轴位置
        QSort(L, low, pivotloc-1); //对左子表递归排序
        QSort(L, pivotloc+1, high); //对右子表递归排序
    }
}
```

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
    L[0] = L[low]; //用子表的第一个记录作为枢轴记录
    pivotkey = L[low]; //枢轴记录关键字保存在pivotkey中
    while (low < high) //从表的两端交替地向中间查找
    {
        while (low < high && L[high] >= pivotkey) --high;
        L[low] = L[high]; // 将比枢轴记录小的记录移到低端
        while (low < high && L[low] <= pivotkey) ++low;
        L[high] = L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low] = L[0]; //枢轴记录到位
    return low; //返回枢轴位置
}
```

主函数会递归的对枢纽的左右边继续进行快速排序

每一趟排序完成, 枢纽元素都会到达最终位置

存储内容	49	27	38	13	49	76	97	65	49
数组下标	0	1	2	3	4	5	6	7	8

pivotloc=4

第一趟排序结束



交换排序快速排序



主函数会递归的对枢纽的左右边继续进行快速排序
每一趟排序完成，枢纽元素都会到达最终位置



交换排序快速排序



主函数会递归的对枢纽的左右边继续进行快速排序
每一趟排序完成，枢纽元素都会到达最终位置



交换排序快速排序

```
void QSort(SqList &L, int low, int high)
{
    //调用前置初值 : low=1; high=L.length;
    // 对顺序表 L 中的子表 L[low..high] 进行快速排序
    if (low < high)
    {
        //长度大于1
        pivotloc = Partition(L, low, high);
        //将L[low..high]一分为二, pivotloc是枢轴位置
        QSort(L, low, pivotloc-1); //对左子表递归排序
        QSort(L, pivotloc+1, high); //对右子表递归排序
    }
}
```

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
    L[0] = L[low]; //用子表的第一个记录作为枢轴记录
    pivotkey = L[low]; //枢轴记录关键字保存在pivotkey中
    while (low < high) //从表的两端交替地向中间查找
    {
        while (low < high && L[high] >= pivotkey) --high;
        L[low] = L[high]; // 将比枢轴记录小的记录移到低端
        while (low < high && L[low] <= pivotkey) ++low;
        L[high] = L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low] = L[0]; //枢轴记录到位
    return low; //返回枢轴位置
}
```

平均情况下空间复杂度 $O(\log_2 n)$
最好情况下空间复杂度为 $O(\log_2 n)$
最坏情况下 $n-1$ 次递归, 空间复杂度 $O(n)$

平均情况下时间复杂度 $O(n \log_2 n)$
最好情况下时间复杂度为 $O(n \log_2 n)$
最坏情况时间复杂度 $O(n^2)$

不稳定!

13	27	38	49	49	65	76	97
----	----	----	----	----	----	----	----



交换排序快速排序

```
void QSort(SqList &L, int low, int high)
{
    //调用前置初值 : low=1; high=L.length;
    // 对顺序表 L 中的子表 L[low..high] 进行快速排序
    if (low < high)
    {
        //长度大于1
        pivotloc = Partition(L, low, high);
        //将L[low..high]一分为二, pivotloc是枢轴位置
        QSort(L, low, pivotloc-1); //对左子表递归排序
        QSort(L, pivotloc+1, high); //对右子表递归排序
    }
}
```

```
int Partition(SqList &L, int low, int high)
{
    // 对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
    L[0] = L[low]; //用子表的第一个记录作为枢轴记录
    pivotkey = L[low]; //枢轴记录关键字保存在pivotkey中
    while (low < high) //从表的两端交替地向中间查找
    {
        while (low < high && L[high] >= pivotkey) --high;
        L[low] = L[high]; // 将比枢轴记录小的记录移到低端
        while (low < high && L[low] <= pivotkey) ++low;
        L[high] = L[low]; // 将比枢轴记录大的记录移到高端
    } //while
    L[low] = L[0]; //枢轴记录到位
    return low; //返回枢轴位置
}
```

每次选择枢轴元素时, 理想状态下是做到最平衡的划分, 即左右两个子问题大小都 $\leq \frac{n}{2}$, 此时时间复杂度为 $O(n \log_2 n)$

平均情况下运行时间与最佳情况下运行时间接近

最坏情况是元素已整体有序或基本逆序时, 时间复杂度为 $O(n^2)$